Identifying the Nutritional Score of a Prepared Meal

Good Old Haggis

https://github.com/dmml-heriot-watt-2025-26/group-coursework-good-old-haggis

Arne Jacobs – H00508832
Ali Amir – H00510041
Hannah Davidson – H00348977
Karol Loskot – H00520109
Wajid Ali – H00523148

Introduction

Promoting healthier eating has become a public health priority. Providing people with a simple visual indicator such as the Nutri-Score is a key component of this effort. Past research has shown that this letter-and-colour-based indicator significantly improves consumer's ability to choose healthier products [1]. Despite its usefulness, the Nutri-Score is limited to packaged foods with nutrition labels leaving a big gap on unlabelled items, restaurant dishes and homemade meals.

This is where artificial intelligence comes in. Recently there have been advances in estimating nutritional content from images. For example, NYU Tandon researchers made a deep-learning system that turns pictures into nutritional analysis by identifying food items [2]. Another study leveraged segmentation and regression for food nutrient estimates [3]. These two and the older MIT's Pic2Recipe project – which used a neural network to return a recipe for a dish based on an image [4] – show that food images can be analysed to give us their ingredients and the nutrients. With this in mind, we aim to build models that map nutritive values of packaged foods to their Nutri-Scores to allow us to predict scores for unlabelled products. We would then extend this to predict Nutri-Scores for homemade meals based on their photos and assess how different combinations of different approaches affect the performance.

In respect to these aims we formulate two research questions:

1. How accurately can we predict the Nutri-Score of a product based on its nutritive values? We expect that training models on data of labelled products will give good results on unscored items.
2. Can we predict Nutri-Scores for homemade meals based on a photo or its ingredients? We expect that ingredient-based predictions would produce better results than image-based predictions but that a multimodal approach would be even better.

By developing these models, we hope to estimate scores for unlabelled items so that customers can make healthier choices and to enable people to find out how healthy they eat by providing scores based on their meal's picture or ingredients.

Related Work

Recent advances in computer vision and deep science enabled progress in food analysis from images. NYU Tandon's research breaks down the nutritional value of a meal with a simple photo combining object detection with portion estimation. A YOLOv8 model identifies the food items and uses its estimated volume, density and area on the picture to predict its size. The training on a dataset of 95,000 images for 200+ food categories provides good results showing it can be used for real-time nutritional assessment. However, it shows us that volume estimation from a 2D image is a real challenge and this project doesn't translate into a nutritional score. [2]

The second research focuses on segmenting the images prior to nutrient prediction. It proposes a framework that isolates food regions in RGB images to extract features and predict macronutrients through a regression model enhanced with attention to channels. They used the Nutrition5k [5] dataset which contains multiple pictures and photos per meal but also ingredients and nutrients and achieved 83% accuracy demonstrating the utility of segmentation. However, they rely mainly on controlled images and predict nutrients without consumer-friendly indicators. [3]

The older MIT's Pic2Recipe model learns a joint embedding between images and textual recipes from Recipe1M – a dataset containing 13M food images and 1M recipes [6]. This enables ingredient inference and recipe prediction (with

65% accuracy) from a single photo, but it doesn't give any quantities which is information needed for analysing the meals healthiness. [4]

Beyond these studies, others have examined food recognition, segmentation, ingredient detection, and nutrient estimation using different models and datasets. Recent works use multi-label ingredient prediction [7], transformer-based models for direct nutrient estimation [8] and multimodal learning for combining visual and textual information [9]. However, most of them work in controlled environments and aren't focused on providing peope with indications. This is the gap that we try to address in our project, providing people with a Nutri-Score for their home-made meals by using a picture.

Dataset Description and Analysis

Both datasets used in this project came from Kaggle. The first one, *Food Ingredients and Recipe Dataset with Images*, is licensed CC BY-SA 3.0 and contains 13,300 prepared meals. There are five columns: *Title*, *Ingredients*, *Instructions*, *Image_Name* and *Cleaned_Ingredients*, providing the necessary information to predict Nutri-Scores based on images or ingredients of a meal. Although the dataset is large, diverse and complete, the ingredient list is highly inconsistent with for example multiple quantities of different units, and contains noise made by unnecessary descriptions. Thus, this dataset required extensive preprocessing before being able to support quantitative modelling for predicting scores based on ingredient composition.

The second dataset is the *Open Food Facts* (Licences: Database ODbL 1.0, Contents DbCL v1.0). It provides 350,000 food products with 163 features and detailed nutritional information. The main features of interest were *product_name*, i*ngredients_text*, *additives_n*, *nutrition_grade_fr* – which represents the Nutri-Score – and nutritional values such as *sugars_100g*. While this dataset has all necessary information to predict the Nutri-Score, there are also some challenges that need to be addressed, such as the high amount of incomplete data and class imbalance in grades.

Together these two datasets make up for a solid foundation for our goals, one possessing ingredient lists and images for meals and the other Nutri-Scores and nutrients of a variety of products, i.e. ingredients.

To prepare the first dataset for analysis, we first loaded the CSV file into a DataFrame with Pandas and removed all columns except *Cleaned_Ingredients.* We then converted it into a NumPy array and added a *meal_id* feature for later linking with the initial data and the images. We removed all meals with more than 12 different ingredients after calculating an average of 11 ingredients per meal. To further reduce the data and outliers we removed all meals with ingredients that have more than 80 characters. The number of meals and ingredients was now small enough to construct – without Kernel Crashes – a tabular of meals (rows) and ingredients (columns). To create this, we captured the different quantities and units (e.g. 1 (3½–4-lb.) or 2¾ tsp.) using utility functions and converted them into grams for consistency. We also removed common descriptors such as "whole", "small" and "chopped", stripped text contained in parentheses, and removed punctuation. We then removed all meals with unique ingredients to reduce sparsity. The greatly reduced number of features enabled manual inspection and led to an endless list of similar ingredients (e.g. *tomato*, *tomatoes*, *diced tomatoes*, *fresh tomatoes)* to merge and non-comestible ingredients (e.g. *6x6 pan* or *wooden skewers*) to remove. Using the *meal_id* we cross-checked cleaned ingredients with the original ingredient lists to ensure accuracy and identify remaining issues. Finally, we normalised ingredients quantities so that each meal's ingredients summed to one. This produced a normalised dataset of comparable ingredient profiles which we saved as a CSV file to use for downstream analysis and model training. During the cleaning process we used AI assistance because of the highly complex nature of the ingredient lists (see Appendix 1).

We began pre-processing the Nutri-Score data by removing all redundant features directly from the dataset as to allow faster reading and altered it to fix formatting issues. We manually removed non-numerical and unwanted features. We observed that Nutri-score values varied based on country, our team needed to select a single output feature to avoid label noise. Our team selected the labelling from the United States as it had the most data and good variation. Following this, we examined the remaining features and noted that a lot of them lacked data entries. We decided to remove all features with less than 70% completion. Then, we identified all non-numerical features and deleted them as they were not necessary for the intended task. Finally, we removed all entries with incomplete fields, the data's size was large enough to do so and would further reduce the processing time when analysing and developing our models.

Once a set of potential features was identified, we performed feature correlation analysis and created a visualisation to allow for easier inspection. We observed that most of our features are independent which is ideal for model building as it will reduce redundancy and allow for better model generalisation. However, Salt-Sodium correlation was '*1*' so naturally one feature was removed (Sodium). We also identified that Fat to Saturated Fat had a correlation of '*0.69*'. Due to their similarity in nature, we later investigated their effects on the model but kept both for the time being. There was an issue when reading the '*ingredients_from_palm_oil_n*' feature so it was removed although if given more time we would have aimed to fix the issue and investigate its impact on model building (See Figure 6).

To investigate the structure, we employed clustering to see how the data separated, given our classification problem. We first reduced the sample size to 50,000 to allow for a better run time. To determine the number of clusters we calculated the silhouette score. The clustering was developed using KMeans. We deemed a range of 2-9 adequate. The best solution was a silhouette score of 0.2945 with the cluster amount equal to 8, implying weak clustering. We then visualised the data to gain further judgment. The data was reduced to 2-dimentions using PCA and plotted. From this plot we interpreted that there is group formation however, there is a substantial amount of overlap. Group 5 (grey) is dominant and spans over a large continuous region, however there are some dense clusters. From this plot you can also depict there are a few outliers in the bottom right region. Overall, the data does not separate that well and is continuous (See Figure 7).

We then selected the most optimal features using the wrapper method paradigm utilising RFECV function. We used Random Forrest (default parameters) as our model due to its ability to handle non-linear relationships, general stable performance and it being a strong selector method by nature. It is also less computationally expensive to run compared to MLP. We required a minimum of 3 features to be selected. We cross verified the results with 3 folds for each permutation based on accuracy. The algorithm determined 7 to be the optimal number, reducing the chance of overfitting. From the selected features we observed that both fat and saturated fat were selected, as mentioned previously, these features are correlated and similar in nature. We hypothesised removing one may improve accuracy and reduce overfitting. We ran tests using the newly selected features with 3 sets: *default*, removed '*fat_100g*' and removed '*saturated_fat_100g*' removed. We recorded that the removal of '*fat_100g*' improved accuracy.

Experimental Setup

Nutri-Score Prediction

The silhouette score influenced the chosen models for the Nutri-Score dataset, as it showed that the data did not form well-separated clusters and had a lot of overlap, which should be taken into consideration when choosing appropriate classifiers. All Nutri-Score classifiers take numerical features as parameters to predict a Nutri-Score from A to E, which are represented by numbers 0 to 4 for the Multi-Layered Perceptron as it did not support a categorical value for the output.

The Random Forest algorithm was chosen as a classifier for predicting Nutri-Scores as this algorithm handles overlapping data well. The Random Forest algorithm also has good balance when handling mid-linear structure, is robust to noise and outliers, and works well with continuous data. We hypothesise that the Random Forest algorithm will perform very well with this dataset.

The Perceptron model was chosen as a classifier for predicting Nutri-Scores as a baseline to measure the success of the Multi-Layered Perceptron, as they are similar in nature. Since the data is not linearly separated and the clusters overlap, we hypothesise that the Perceptron model will not perform as well as the other models.

The Multi-Layered Perceptron model was chosen as a classifier for predicting Nutri-Scores as there is a large amount of complex, overlapping data to process, which can be solved using a Neural Network. Hyperparameter tuning would have a large effect on the model's accuracy, however this model could be susceptible to overfitting as it can easily be made too complex for the data. We hypothesise that this model will be successful but may not perform as well as the Random Forest.

One performance metric used for the Perceptron and MLP models was training vs testing accuracy, as comparing the two can suggest whether a model is overfitted, and both give a general idea of how successful the model is at learning and predicting attributes and results. Both models also use precision, recall and F1-score as these give valuable insight into how many positive predictions per class were truly positive, and how many positive instances of each class were predicted as true positives, with the F1-score averaging both scores. This also helps further visualise accuracy on unbalanced data, where the model may end up better at predicting a more common Nutri-Score.

Cross-validation was also used to measure the performance of the MLP model, as it also gives insights into unbalanced data by training and testing the model on different folds of the set. This can show the effects of unbalanced data on the model's performance and combats the issue of any randomly selected testing set being unbalanced and not indicative of the model's true accuracy.

The Random Forest model also uses training vs testing accuracy, precision, recall and F1-score to measure performance and accuracy. We also used Stratified K-Fold Cross-validation, which was chosen as it balances the proportions of data in each fold for more consistent and accurate cross-validation.

Meal Classification

K-Means Clustering was chosen as a classifier as it is efficient and interpretable and can be used to group meals that have similar ingredient patterns into distinct categories, which act as classes for the CNN. We also had to use clustering as a metric to check our dataset quality after each iteration of standardization, and being computationally efficient, it allows quick experimentations. We hypothesise that K-Means would group together meals that share similar ingredient components. However, due to the high dimensionality and the imbalance of the dataset, the clusters would not be perfect.

After running K-means for a various number of clusters and visually analysing them, it made sense to have the range in between 15-20, since the dataset contained food from various cuisines and had a variety of ingredients. To justify this, we did silhouette-based validation. The silhouette scores were computed for values of k ranging from 2-25. 18 gave the highest silhouette score, and it gave well defined yet diverse clusters. (See appendix 2 for cluster distribution).

The CNN algorithm was chosen as an image classifier for the Recipes dataset, for learning visual representations and classifying the images of different meals into classes created by K-Means. This is because the CNN algorithm can identify features of images using convolutional layers. We hypothesise that it will learn visual characteristics of the images and identify how the food looks, but the accuracy might be low because the cluster labels were generated from non-visual

features, and many visually similar meals may belong to different ingredient groups. The performance of each CNN is judged by training and testing accuracies, F1, recall, and precision scores, along with the confusion matrices.

So, after standardizing the dataset, each row represents the ingredient compositions of the respective meal. The meals are passed through the K-Means algorithm that categorizes them into different clusters, which are defined as labels for the CNN. Finally, the CNN takes the food images as inputs and try to predict which ingredient label do those meals fall into.

Further Tuning

Hyperparameter tuning was performed on the Random Forest model to maximise accuracy, since it is the best performing model, which resulted in an improvement of 0.26.

The Keras MLP model appeared very overfitted with training accuracy at 99% at its highest, while testing accuracy was 35%. Switching to the Scikit implementation of MLP gave better results with predictions initially at 79% accuracy. Initially the MLP algorithm was tested with 100 neurons in one hidden layer, before different hyperparameters were tested to improve the model further, using Scikit's GridSearch. The best parameters found were one layer with 75 neurons, and another with 50 neurons. This improved the final accuracy on the testing data to 80%, which was an 0.9% increase. Earlier runs of the CNN model were overfitted and had an unstable loss pattern. Adding batch normalization added more stability. Adding global average pooling also improved the model, since it reduced the parameters. Since the dataset size was not huge enough, having a dense layer of 256 was also leading to overfitting, so it was replaced by 128 which gave better results. We started with a learning rate of 0.0003 but the accuracy was plateauing very quickly. To solve this, we added ReduceLROnPlataeu to dynamically change the learning rate while training. We also rescale the images so that all pixel values are in between 0 to 1. Stratify was also added to split the training and testing data evenly among all clusters.

Having unbalanced cluster sizes was giving a very biased confusion matrix for CNN. For our 3rd version of CNN, a problematic cluster was removed, and the rest of them were augmented to 100 images each, which significantly improved fairness across classes.

For clustering the meals, we normalized each row such that each meal is represented as a vector of ingredient proportions that add up to 1. For improving clustering results and decreasing dimensionality, we filtered the ingredients that were present in less than 0.1% and more than 90% of the meals, which reduced some overlap. 85 of the total 289 ingredients were used for clustering.

Results

Nutri-Score Prediction

The classification models were evaluated using the feature set determined through pre-processing for model comparison. The Random Forrest model performed the best with highest accuracy being 0.9505 after applying 10-fold cross validation and a standard deviation of 0.0008 indicating a well performing and stable model. The next best performing model was the MLP with best accuracy score being 0.8722 after 10-fold cross validation however the results produced a standard deviation of 0.0443. The model has good accuracy however the cross-validation results vary significantly, implying that model stability can be improved. Perceptron performed the worse with an accuracy score of 0.68. Overall, the models performed as we hypothesised although we did not expect such a significant difference between MLP and Perceptron models due to their similar nature. Other metrics were used for further insight into precision, recall and how our models handle class imbalances as well as a visualisation of confusion matrices (see Figure 10,11,12) to understand the

performance of each class prediction (see appendix 8). Predictions for category B, C proved least accurate across all models with f1-scores being ~0.04 less than counterparts, given more time we would investigate further to tune better.

Meal Classification

The K-Means provided satisfactory results (see appendix 2,3). While it was able to group meals together that had similar ingredients, all the meals that did not have a strong enough composition of any ingredient were cluttered together close to the origin. This noisy cluster was causing problems for the CNN because it did not have much impactful information, and being the largest in size, it caused imbalance between the clusters (see appendix 4). This was expected due to the high dimensionality of the dataset, and we explore its effect further in the during the CNN experiments. The CNN's started with a basic 3-layer convolutional structure, and the first runs displayed extreme overfitting, which was soon minimized by reducing the dense layer size and introducing batch normalization and global average pooling at each step between the fully connected layers. Having a deeper architecture also resulted in overfitting, because our dataset size was small. Furthermore, due to the noisy cluster, the model was making almost all of the predictions only for that class. To introduce balance, we deleted the noisy cluster and augmented the rest of the classes to have 100 samples each. This increased our F1, recall and precision scores from 0.9, 0.10, 0.19 to 0.41, 0.43, 0.41 respectively and a more balanced confusion matrix (see appendix 7). So, after these experiments, we were at the training accuracy of 50%, and 43% test accuracy. These support our hypothesis, that having a visual classifier is going to struggle for labels based on textual ingredient data only, because of the ambiguities between the dataset (see appendix 5). To further support this, we also tried a concatenated model that landed at around an 80% accuracy (see appendix 8). Detailed descriptions of all these experiments are at (appendix 10).

Finally, to bridge our two modules, we mapped the ingredient composition of the clusters to the macro values for Nutri-score predictions. (See appendix 11 for detailed results)

Discussion

Nutri-Score Prediction

We found that two of our chosen models, the Random Forest and the Multi-Layered Perceptron, were able to accurately predict the Nutri-Score of a food based on its ingredients. This gives good results as we hypothesised, since all components are present and accurate in quantity, leading to a semi-linear function based on nutritional contents.

During our experiment, feature selection was performed once with a wrapper function using a Random Forest algorithm, and the selected features were used for developing each final model. In the future, this feature selection should be performed on each individual model to identify which features affect each model the most and ensure more accurate results. Future research should also focus on balancing the features to prevent the model becoming biased towards more common categories, as some models, especially the Perceptron, struggled to predict B and C scores more than the others (see Figure 10).

Meal Classification

The clustering stage for the meals produced mixed results. While K-Means succeeded in grouping many similar ingredient meals together, even the best silhouette score (0.2989 at K=18) was relatively low, which shows that many meals did not have distinct compositional patterns. This comes in line with the occurrence of a large cluster containing all the meals with

weak ingredient compositions, that created noisy class labels for the CNN and degraded its performance. The CNN also struggled as it trained on the image pixels but had to classify on labels that were entirely made from ingredient compositions. So, in reference to our research question, we cannot really predict accurate Nutri-score of homemade meals from just the images alone without including the ingredients, unless the dataset is detailed enough to train the models accordingly. This was to be expected, as prior research on image classification had 65% accuracy [4], or 83% accuracy with the support of ingredient and nutrient data [3]. For future implementations, using the image vectors during the clustering process might be a good way to go. Future research should also look using pre-trained deep CNN's to more accurately extract features from images which can be used to categorize them.

Real-World Implications

Overall, the models implemented are robust with few errors that cannot be amended with more investigation or tuning, such as noisy datasets or more detailed feature selection. The full pipeline is not realistic in a practical application as the image classification accuracy is too low to give valuable real-world results, however the classification of the ingredient clusters and Nutri-scores is reliable and accurate, and could be used to give further insight into the nutritional value of existing recipes, which could aid research into the nutritional quality of food and promote healthy eating for consumers if implemented as some form of commercial application.

This project proved that it was possible to analyse different components of a meal to predict the Nutritional Score of the meal, which builds upon the existing image classification problems we researched by implementing a method to predict detailed nutritional information and nutritional quality from image classification results.

Conclusion

Public health and awareness of healthy eating continue to be important matters in the modern day. To tackle the issue of awareness of how healthy our meals really are, we selected two datasets to perform research on: the *Food Ingredients and Recipe Dataset with Images* dataset, which contains valuable information about prepared meals and which ingredients they contain, and the *Open Food Facts* dataset, which contains information about how nutritional information correlates to a product's Nutri-Score.

We chose a combination of Convolutional Neural Networks and K-Means clustering to predict a food cluster that an image of a meal would belong to, and which macros were likely to be in that food based on the cluster it was assigned to. We chose the Random Forest and Multi-Layered Perceptron algorithms to predict the Nutri-Score of a product based on its macros, and the Single-Layered Perceptron as a baseline to compare to the MLP algorithm.

Overall, we were able to predict the nutritional values of a meal based on an image of the meal, although it did not have high accuracy as images of food alone do not contain accurate and detailed information about the nutritional composition. However, the clustering based on ingredients was accurate once outliers were considered and removed. We were able to accurately predict the Nutri-Score of a food based on its ingredients using models more suited to complex functions, as the relationship between ingredients and Nutri-Score is semi-linear, although this meant that the single Perceptron did not perform as well. By combining the predictions on the datasets, we were able to successfully predict a meal's ingredients and nutritional information based on an image, then predict the Nutri-Score based on the ingredients. These models could be improved with further tuning, better feature selection and less noisy datasets for use in a practical application, which could aid research into the nutritional quality of food and promote healthy eating for everyday consumers.

References

[1] IARC (from WHO), The Nutri-Score: A Science-Based Front-of-Pack Nutrition Label Helping consumers make healthier food choices, 2021.

https://www.iarc.who.int/wp-content/uploads/2021/09/IARC_Evidence_Summary_Brief_2.pdf

[2] P. Panindre and S. Kumar, “Ai food scanner turns phone photos into nutritional analysis,” NYU Tandon School of Engineering, 2025.

https://engineering.nyu.edu/news/ai-food-scanner-turns-phone-photos-nutritional-analysis

[3] Y. Zhao, P. Zhu, Y. Jiang, and K. Xia, “Visual nutrition analysis: leveraging segmentation and regression for food nutrient estimation,” Frontiers in Nutrition, 2024.

https://www.frontiersin.org/journals/nutrition/articles/10.3389/fnut.2024.1469878/full

[4] A. Conner-Simons and R. Gordon, “Artificial intelligence suggests recipes based on food photos,” MIT News, 2017.

https://news.mit.edu/2017/artificial-intelligence-suggests-recipes-based-on-food-photos-0720

[5] Nutrition5k: A Comprehensive Nutrition Dataset

https://github.com/google-research-datasets/Nutrition5k

[6] MIT, Recipe1M+: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images, 2017

https://im2recipe.csail.mit.edu/

[7] Kun Fu & Ying Dai, “Recognizing Multiple Ingredients in Food Images Using a Single-Ingredint Classification Model, https://arxiv.org/pdf/2401.14579

[8] Z. Kwan, W.Zhang., et al., NuNet: Nutrition Estimation for Dietary Management: A Transformer Approach with Depth Sensing, 2024

https://arxiv.org/pdf/2406.01938

[9] Yuehao Yin, Huiyan Qi, Bin Zhu, et al., FoodLMM: A Versatile Food Assistant using Large Multi-modal Model, 2024 https://arxiv.org/pdf/2312.14991

Appendices

1. Example of the first meal's ingredients:

array(['[\'1 (3½–4-lb.) whole chicken\', \'2¾ tsp. kosher salt, divided, plus more\', \'2 small acorn squash (about 3 lb. total)\', \'2 Tbsp. finely chopped sage\', \'1 Tbsp. finely chopped rosemary\', \'6 Tbsp. unsalted butter, melted, plus 3 Tbsp. room temperature\', \'¼ tsp. ground allspice\', \'Pinch of crushed red pepper flakes\', \'Freshly ground black pepper\', \'⅓ loaf good-quality sturdy white bread, torn into 1" pieces (about 2½ cups)\', \'2 medium apples (such as Gala or Pink Lady; about 14 oz. total), cored, cut into 1" pieces\', \'2 Tbsp. extra-virgin olive oil\', \'½ small red onion, thinly sliced\', \'3 Tbsp. apple cider vinegar\', \'1 Tbsp. white miso\', \'¼ cup all-purpose flour\', \'2 Tbsp. unsalted butter, room temperature\', \'¼ cup dry white wine\', \'2 cups unsalted chicken broth\', \'2 tsp. white miso\', \'Kosher salt\', \'freshly ground pepper\']', ...])

Problems:

Quantities: "1 (3½–4-lb.)"

- "1" for one whole chicken which makes access to the real weight extremely hard because we can't simply remove the first numbers of each ingredient as these are most of the time the needed number.
- "3½–4-lb." hard to access but also hard to correctly analyse as it has two values.

Units: "lb.", "tsp.", "Tbsp.", "cups", "oz.", "cup"

- For some ingredients we encounter different units across the dataset which makes them hard to extract and normalise
- Converting everything to grams helps but brings another problem which we did not tackle. Different ingredients have highly different density. Thus, converting 1 cup of milk or 1 cup of flour to grams gives the same values although in reality thay ar very different 240g vs 120g.

Ingredients without units or quantities:

- "Pinch of ...": unit without number this hard to quantify
- "2 small acorn squash": value but no unit thus this would require manual conversing to grams depending on the ingredient
- "Kosher salt": no value nor unit, thus no way to know the quantity

Long ingredients with noise because of unnecessary details: "6 Tbsp. unsalted butter, melted, plus 3 Tbsp. room temperature"

- "melted" and "room temperature" for example
- We also have another problem with the addition of two "types" of butter in one string as this makes it difficult to capture both. Our code would only capture the "6 Tbsp."

Because of all these factors we decided to use ChatGPT to assist us in the cleaning of the dataset, but the result is still far from being perfect. On the Teacher Assistants' recommendation, we asked ChatGPT to return a cleaned CSV file, but all the attempts gave worse results than the one our code gave us.

The complexity of the preprocessing made it so that the use of ChatGPT was basically going back and forth. We would ask for something, it would only partially work for our data, thus we would modify it slightly ourselves and for bigger problems ask for a corrected version. As there were problems appearing at every step this back and forth lasted for the whole preprocessing, and it would still go on as the data is still not perfect.

Thus, the code for the preprocessing is entirely a mix of ChatGPT and our handwritten code or modifications. Here are the links of our exchanges if you want to verify more in detail:

https://chatgpt.com/share/691e8dd8-2760-8006-991f-af43e67d70aa

https://chatgpt.com/share/691e8e0b-49f0-8006-ab3d-b6d6d2642358

2. Meals per Cluster:

Clusters	**No of meals per cluster**
Cluster 0	50 meals
Cluster 1	29 meals
Cluster 2	77 meals
Cluster 3	430 meals
Cluster 4	83 meals
Cluster 5	40 meals
Cluster 6	52 meals
Cluster 7	40 meals
Cluster 8	25 meals
Cluster 9	88 meals
Cluster 10	35 meals
Cluster 11	67 meals
Cluster 12	55 meals
Cluster 13	45 meals
Cluster 14	13 meals
Cluster 15	24 meals
Cluster 16	8 meals
Cluster 17	108 meals

Table1: Cluster Distribution

Average cluster size: 70.5 and standard deviation 93.656

3. Example ingredient composition and meals:

Here are some of examples of the clusters, the most common ingredients in those clusters, and some of the meals that were categorized by the clustering algorithm.

Cluster 11 (67 meals):

Top ingredients composition:

- Milk: 0.5179570747082176
- Egg: 0.10924151807196059
- Sugar: 0.07837715068863338
- Flour: 0.04239929886790481
- Butter: 0.03719036809255065
- Cream: 0.03541488977566882 -
- Chocolate: 0.017058991210495127

Example meals:

- Tangy Frozen Greek Yogurt
- Chocolate Cream and Cinnamon Sugar
- Lemony Yogurt Sauce
- Iced Coffee Shakerato
- Baked Egg Custard with Gruyère and Chives

Cluster 15 (24 meals):

Top ingredient composition:

- Bourbon: 0.6863687744645048
- Vermouth: 0.09859214829418732
- Water: 0.046970247878686804
- Campari: 0.036007559958289886
- simple syrup: 0.026551417692196

Example Meals:

- Orange Mint Julep
- Perfect Manhattan
- Manhattan
- Sazerac
- The New York Sour
- Scotchy Boulevardiers for a Crowd

4. The “problematic” cluster:

After multiple clustering runs for different values of K, there is always one cluster, very close to the origin that gathers the largest number of meals. A t-SNE plot is shown as figure 1. The reason behind the occurrence of this cluster is that it contains all the meals that have a variety of ingredients, but not a strong enough composition for any of them. Hence, the meals are not assigned to a distinct ingredient cluster and are rather cluttered together. This causes problems for the CNN for two reasons:

- Since there is not much useful information in that cluster for the CNN to learn, it acts as noise.
- Being the largest cluster and having a sample size of 430, this class is the main reason for the dataset imbalance, as the average cluster size is 70.5 meals.

To support this, here are the top ingredients and their composition values for this cluster:

- Water: 0.039366
- Nuts: 0.034841
- Wine: 0.028633
- Sugar: 0.02706
- Oil: 0.026099
- Egg: 0.021910
- Beef: 0.021576
- Pasta: 0.019967
- Salt: 0.019916
- Mayonnaise: 0.01957
- Butter: 0.01927
- Onion: 0.018296
- Shrimp: 0.014115

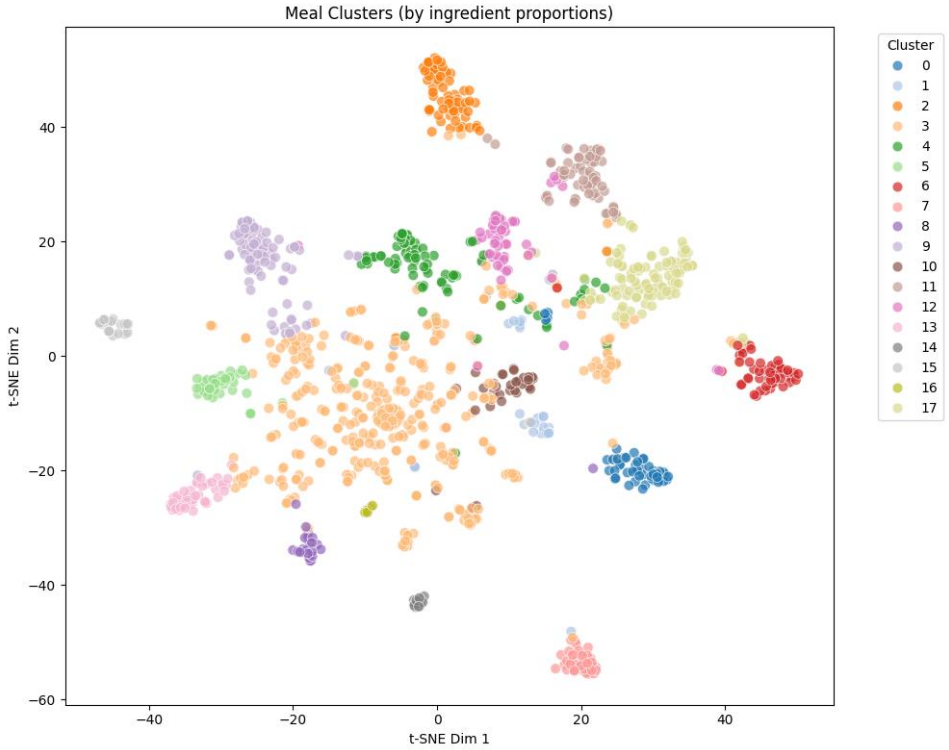

Figure1: T-SNE plot of K-Means Clustering of meals.

5. Visually similar meals grouped into different clusters:

Since we are clustering based on ingredients, if a pasta and a pizza contain similar ingredients, they are grouped into a single cluster. Alternatively, to our disadvantage many similar looking meals that contain different ingredients are assigned different clusters. Here are some examples:

Figure 2 shows a picture of a berry punch, which is assigned to cluster 6 which has all the food that have high concentration of berries. On the other hand, figure 3 shows a picture of an Absinthe Sazerac which is assigned to cluster 15, which mainly contains alcoholic cocktails. Both these images look very similar, but are composed of entirely different ingredients, and thus are assigned to different clusters. This creates confusion for our CNN, since it learns on visual information only.

Figure 2: Berry Rum Punch

Figure 3: Absinthe Sazerac

Another example is of ambiguous pictures. Figure 4 is an image of “Greek yogurt ice-cream" and is assigned to cluster 11. Figure 5 is entered as an image of a “Microwave caramel sauce” and has the ingredients for the caramel sauce, so it lands in cluster 4. However, the image itself also contains the ice-cream which is perhaps even more prominent than the sauce and resembles figure 4.

Figure 4: Greek yogurt ice-cream

Figure 5: 3-Ingredient microwave caramel sauce

There are countless examples like these in our dataset, which cause confusion for the CNN and degrade the performance.

6. Data Pre-Processing Figures for Nutri-Score Prediction

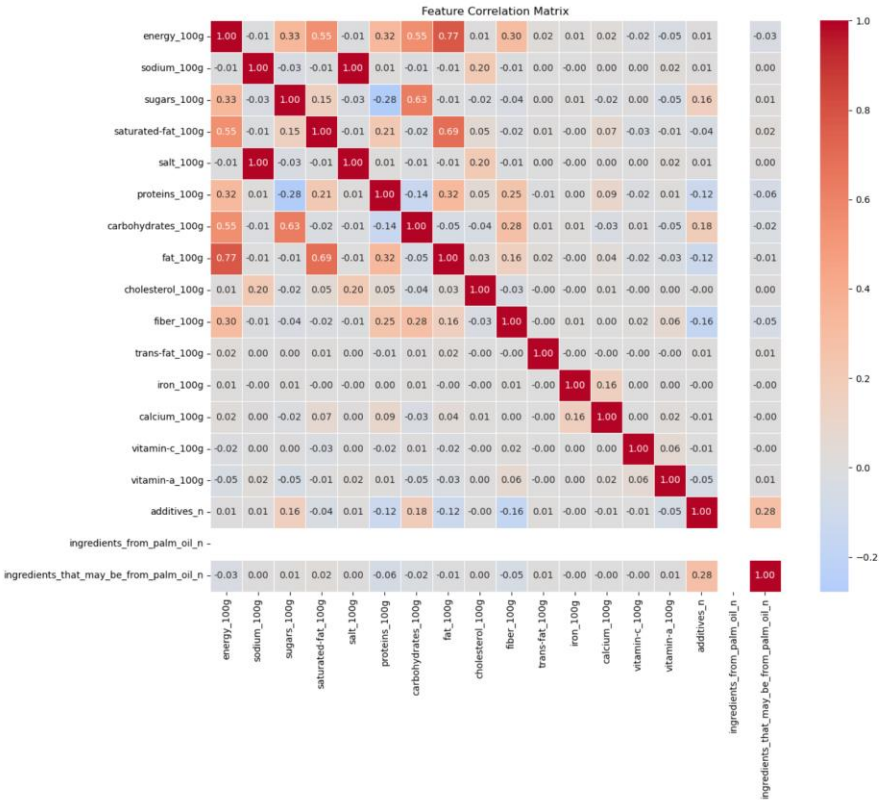

Figure 6: Correlation matrix of potentially selected features

Figure 7: Clustering visualisation for exploratory analysis of Nutri Score

7. Confusion matrices before and after balancing classes:

Figures 8 and 9 show the confusion matrixes with essentially the same CNN architectures, but with different dataset handling. Before removing the problematic cluster and having unbalanced sample sizes, the CNN found one “safe” class and made most of the predictions for that class, whether right or wrong. This was the class with 430 samples. That does not necessarily mean that it learns much from this class but rather made more predictions for that class and a lot of them ended up being true. Most of the classes have no predictions at all.

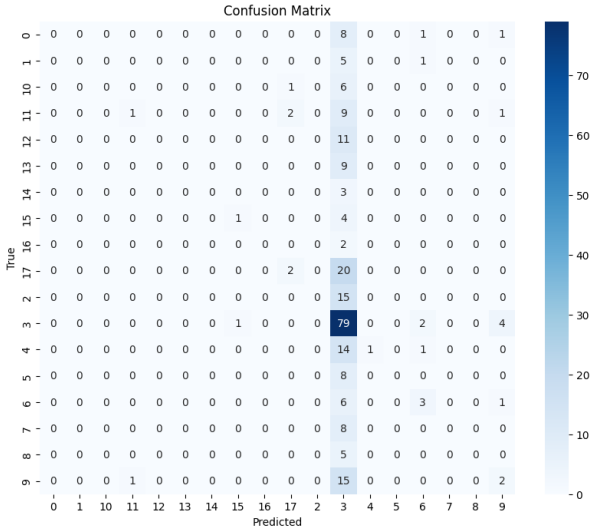

Figure 8: Confusion matrix before balancing clusters

Then we removed the problematic cluster and decided to artificially augment all the other classes to 100 images. AI was used for the image augmentation part, and the code is in the notebook. Since the colour of the meals are an integral part, we did not change the brightness or contrast during augmentation, but mainly sizing shifts, rotations, and positional changes. Figure 9 shows the confusion matrix after these changes, and we can see that the predictions are much more balanced between the classes. The f1, recall, and precision scores also improved after this, and the accuracy rose about 10%.

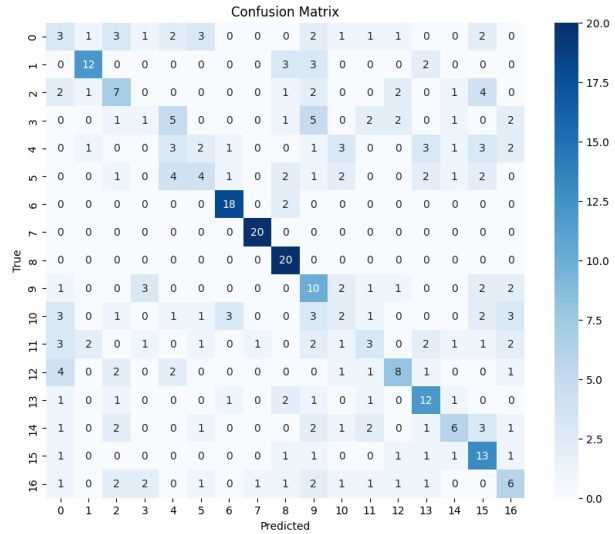

Figure 9: Confusion matrix after removing the biggest class, and augmenting images

8. The concatenated model (generated with AI):

Layer (type)	Output Shape	Param #	Connected to
image_input (InputLayer)	(None, 128, 128, 3)	0	-
conv2d_32 (Conv2D)	(None, 128, 128, 32)	896	image_input[0][0]
max_pooling2d_24 (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_32[0][0]
conv2d_33 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling2d_24…
max_pooling2d_25 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_33[0][0]
conv2d_34 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d_25…
global_average_poo… (GlobalAveragePool…	(None, 128)	0	conv2d_34[0][0]
input_layer_5 (InputLayer)	(None, 289)	0	-
dense_35 (Dense)	(None, 128)	16,512	global_average_p…
dense_36 (Dense)	(None, 64)	18,560	input_layer_5[0]…
dropout_21 (Dropout)	(None, 128)	0	dense_35[0][0]
dropout_22 (Dropout)	(None, 64)	0	dense_36[0][0]
weighted_image_bra… (Lambda)	(None, 128)	0	dropout_21[0][0]
weighted_ingredien… (Lambda)	(None, 64)	0	dropout_22[0][0]
merged_features (Concatenate)	(None, 192)	0	weighted_image_b… weighted_ingredi…
dense_38 (Dense)	(None, 256)	49,408	merged_features[…
dropout_23 (Dropout)	(None, 256)	0	dense_38[0][0]
dense_39 (Dense)	(None, 18)	4,626	dropout_23[0][0]

After trying a couple of different architectures and consistently seeing bad accuracies, I was convinced that it is because of CNN architectures, but because the classes are made up of ingredient elements and the model is learning on the image pixels alone. This leads to unfair circumstances (appendix section 5 further explains this). Which drove me to questioning whether the results would be different if either the image pixels were involved in the clustering process, or the ingredient data was involved in the classification process.

Hence, I decided to try a CNN which also handles the ingredients. The architecture was too complicated for me to code, so ChatGPT was used to build this upon my previous CNN code. I gave my previous CNN structure and explained the idea, and it gave me the code which I have included in the notebook. The idea was to take the ingredient data as an input vector alongside the image pixels and then concatenate these two vectors to have a sort of multi modal approach. The test accuracy climbed to around 90%. At first, I was extremely happy with this result, as it explained why my previous models were having such low accuracies, and supported my hypothesis. However, upon discussing this with the lab helpers, I was told that when two vectors are concatenated in the CNN, it is possible that the model ignores the image pixels entirely and only learns on the ingredient data. Hence, this accuracy could be deceiving.

To counter this, I did try to add a weight balance between the two vectors, and even weaking the ingredient branch by reducing its size from 128 to 64, but the accuracy only dropped to an 80%. Ultimately, keeping the advice in mind, and as we were also unsure about this CNN architecture and the reliability of the accuracy, we decided that this might not be the perfect approach to move forward with.

So, we kept CNN Architecture 3 as our main approach and did not take this experiment any further.

9. Results of models for Nutri-Score predictions

Nutri-score raw runs for model selection

- Random Forrest (pre wrapper method): Training Accuracy: 0.9999, Test Accuracy: 0.9576, Difference: 4.4214%. Very good performance with very minimal overfitting (expected for RF)
- MLP ((100,), (Data normalised, pre wrapper method)): Training Accuracy: 0.9467, Test Accuracy: 0.9400, Difference: 0.7098%. Another good performance with very good generalisation.
- Perceptron (Data normalised, pre wrapper method): Training Accuracy: 0.5353, Test Accuracy: 0.5292, Difference: 1.1598%. Expected behaviour, just better than randomly guessing.

Nutri-score Random Forrest results

- Accuracy: 0.9463 Precision: 0.9467, Recall: 0.9463, F1-Score: 0.9464 (on standardised fold random state = 42)
- Stratified 10-fold CV: Accuracy: 0.9505 SD: 0.0008 (Stable model with great accuracy)

Nutri-Score Perceptron results

- Accuracy: 0.68, Precision: 0.71, f1-score: 0.68

Nutri-Score MLP results

- Initial prediction (one layer of 100 neurons): Accuracy: 0.7985, Precision: 0.81, Recall: 0.80, F1-Score: 0.80
- After hyperparameter tuning (75, 50): Accuracy: 0.8073, Precision: 0.81, Recall: 0.81, F1-Score: 0.81
- 0.9% improvement to accuracy, hyperparameter tuning appears to have improved recall the most.
- 10-fold cross-validation: Accuracy: 0.8722, SD: 0.0443 (very good accuracy, however cross-validation results were varied for each fold, meaning model stability could be improved.)

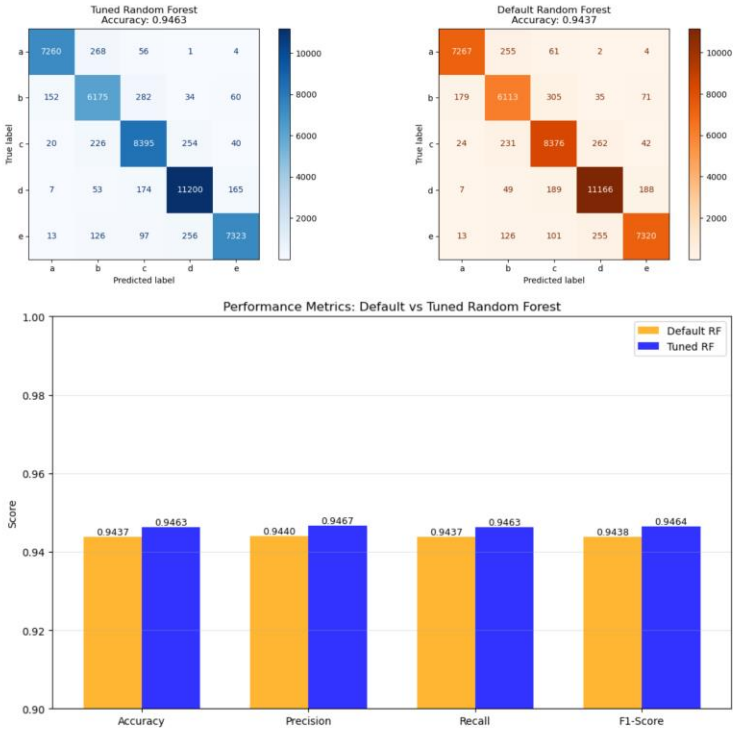

Figure 10: Confusion matrix of Random Forrest comparing performance before and after hyperparameter tuning

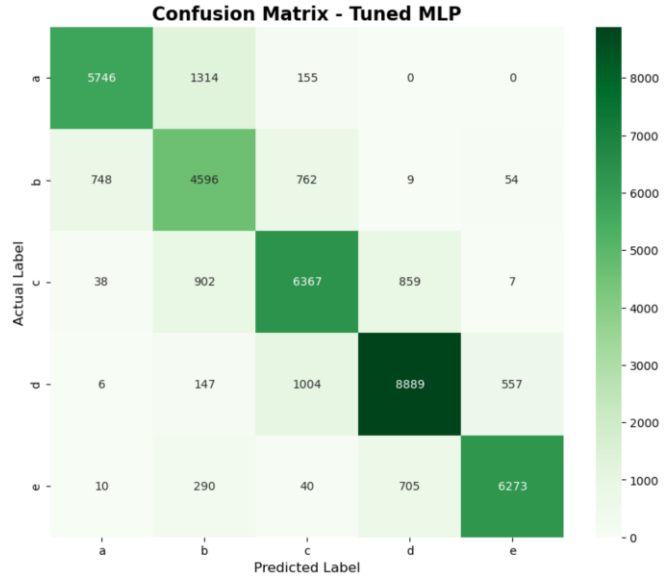

Figure 11 Confusion matrix of Multi-layer Perceptron after hyperparameter tuning

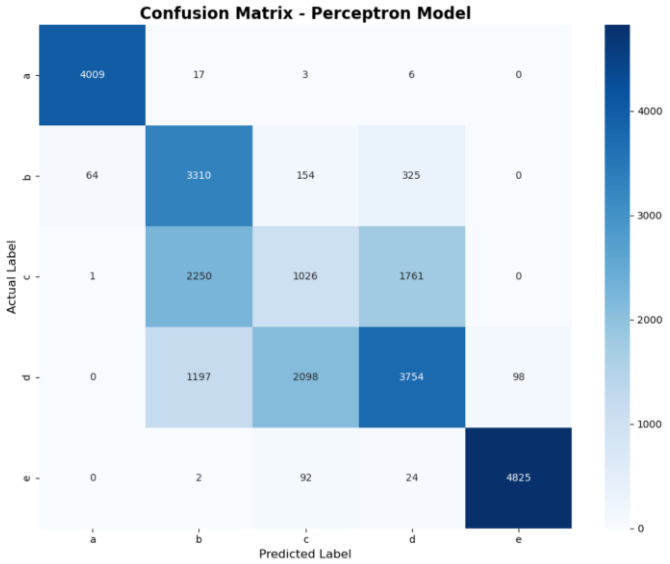

Figure 12: Confusion matrix of Perceptron

10. Results of image models:

Meal-Clustering Result:

- Used k=18 as it gave the highest silhouette score, which still was not great (K=18: 0.2989)
- Worked well for the most part, grouped similar ingredient meals together (See appendix. 2,3)
- Problems arise with a cluster that has absorbed all the meals with no strong ingredient composition and has the largest sample size. (See appendix 4.)
- Many visually similar meals are assigned to different ingredient clusters, which causes problems for the CNN, since it will be trained on visual information only. (See appendix 5.)

CNN Architecture 1:

- A baseline CNN to see the initial performance without any tuning. Built using 256 neuron dense layer, 0.4 and 0.5 Dropouts and has 3 convolution blocks
- Training Accuracy: 84.6 %, Test Accuracy: 29.5%, F1: 0.12, Recall: 0.12, Precision: 0.19
- Model showed severe overfitting. So, decided to take measures to minimize it.

CNN Architecture 2:

- The goal of this model was to minimize the overfitting seen in the previous one.
- Reduced the dense layer to 128 neurons, added Batch Normalization and Global Average Pooling among 3 convolutional blocks. Kept one dropout before the output layer for 0.4.
- Training Accuracy: 36.06 %, Test Accuracy: 35.03%, F1: 0.10, Recall: 0.9, Precision: 0.19.
- Although overfitting was reduced, the classification report and confusion matrix showed poor and highly biased predictions. The model generalizes to one class.

CNN Architecture 3:

- The confusion matrices of the previous runs were very biased towards one class.
- There was a problematic cluster that was catching all the meals that did not have strong compositions of any ingredients and had the largest sample size (430). To explore its effect, we tried removing it and augmented the rest of the classes to 100 samples each for fairness.
- Training Accuracy: 50.76%, Test Accuracy: 43.27%, F1: 0.41, Recall: 0.43, Precision: 0.41.
- Kept a similar architecture from the previous CNN to compare. The accuracies had an almost 10% improvement, but more importantly, the confusion matrix was a lot more balanced among all classes. (See appendix 7).

CNN Architecture 4:

- The goal of this model was to see if adding more convolutional blocks would improve the performance.
- A deeper CNN consisting of 5 convolution blocks with batch normalization and a 64-neuron dense layer. Used the balanced clusters for this as well with 100 samples each.
- Training Accuracy: 82.6%, Test Accuracy: 30.7%, F1: 0.29, Recall: 0.31, Precision: 0.28.
- Due to the relatively smaller size of our dataset, adding more layers reintroduced overfitting.

CNN Architecture 5:

- This model was attempted to support the argument that our CNN's performing poorly was not a result of bad architectures, but rather the classes being made from textual ingredient data, and the CNN's trying to classify based on visual information only.
- A model that also takes an ingredient vector along with image pixels and concatenates both, generated with AI. (See appendix 8 for further discussion)
- Training Accuracy: 96%, Test Accuracy: 81.10%.

11. Results after bridging models:

After obtaining all ingredients compositions for all clusters, we calculated the average quantity of each ingredient for every cluster, using Excel formulas.

Then, we had to map out the macro quantities for all 289 ingredients. For this process, a lot of help from ChatGPT was used. For some ingredients, we found exact matches on publicly available sources such as USDA, for some approximate matches were used (since we had categorized ingredients intro singular groups, for instance, walnuts, cashew nuts, pine nuts to 'Nuts') and for the rest estimated values were used. The excel file is in the project directory and is titled "ingredient_macros.xlsx" and the data looks something like this:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
ingredient	energy_100g	fat_100g	saturated-fat_100g	trans-fat_100g	cholesterol_100g	ohydrates_	sugars_100g	fiber_100g	proteins_100g	salt_100g	sodium_100g	vitamin-a_100g	vitamin-c_100g	calcium_100g	iron_100g	source
absinthe	966.504	0	0	0	0	0	0	0	0	0	0	0	0	0	0	approx→spirit_unsweet (typic
acorn squash	188.28	0.1	0.02	0	0	12	3.5	2	1	0	0.003	0.00011	0.11	0.04	0.0007	approx→veg_squash (typical)
agave nectar	1297.04	0	0	0	0	76	69	0	0	0	0.004	0	0	0.001	0	USDA/MyFoodData (typical); e
agave syrup	1297.04	0	0	0	0	76	69	0	0	0	0.004	0	0	0.001	0	USDA/MyFoodData (typical); e
aged balsamic vinegar	368.192	0	0	0	0	18.1	15	0	0.5	0	0.023	0	0	0.027	0.0007	approx→balsamic vinegar (USI
almond	2422.536	50	3.8	0	0	22	4.4	12.5	21.2	0	0.001	0	0	0.264	0.0037	USDA/MyFoodData (typical); e

After this, these values were multiplied to the ingredient weights for each cluster. Here is the output:

cluster	energy_100g	fat_100g	saturated-fat_100g	trans-fat_100g	cholesterol_100g	carbohydrates_100g	sugars_100g	fiber_100g	proteins_100g	salt_100g	sodium_100g	vitamin-a_100g	vitamin-c_100g	calcium_100g	iron_100g
0	2221.283271	53.77771323	32.62586164	1.901422772	0.145620194	12.75378978	6.260134827	2.175159	2.262222473	3.1514787	1.275924954	0.000456765	0.003282809	0.054812196	0.002431
1	1657.366218	29.00111434	5.58672526	0.1406033	0.029050414	26.12337364	13.45759435	6.63792	10.18504866	0.895681	0.365085085	4.85501E-05	0.012432173	0.134601868	0.002671
2	766.1556685	13.43723201	4.789079295	0.170196789	0.295855494	5.753084681	2.852929368	0.69607	11.17993557	0.5515587	0.222402332	0.00015259	0.001723556	0.064135244	0.001909
3	902.8341831	11.92437858	3.786478929	0.149522822	0.026822821	19.32361464	8.51405827	3.36209	6.055019086	2.4827967	1.001039076	0.000204925	0.012193043	0.070639579	0.002699
4	1346.413215	7.622021789	4.025300418	0.188052622	0.042253337	62.50823878	58.33251898	0.991762	2.058664807	1.1327783	0.473922347	6.29487E-05	0.001136461	0.019151241	0.000822
5	739.4404522	1.394006761	0.394924392	0.002037005	0.004843087	8.570725498	4.827237841	2.009297	1.142827054	1.083147	0.433840369	0.000137441	0.003793829	0.033752942	0.001521
6	552.3447592	2.883020872	1.405445584	0.054793474	0.011930847	25.19840268	19.84886428	3.348726	1.448486425	0.0745129	0.031563456	3.00518E-05	0.146044518	0.02264584	0.000626
7	553.0288232	6.737979118	2.49075243	0.11759519	0.020658691	15.15779192	1.214840096	2.078009	2.83375581	1.3099631	0.525034841	9.17108E-05	0.016137202	0.035252098	0.001019
8	779.6023672	9.596046075	3.439663311	0.194739726	0.057859712	7.410055284	1.250473091	1.361583	18.23459778	0.5681566	0.227501341	5.45268E-05	0.003343201	0.031099472	0.001623
9	329.5321951	3.24486023	1.37713722	0.051649886	0.007131926	8.796414272	5.91058298	0.765918	1.093716002	0.3660814	0.146569956	4.31679E-05	0.0049176	0.023087649	0.000467
10	1493.039541	26.11021616	14.29017128	0.672503283	0.07976294	14.00673722	5.023941206	1.989076	16.1662813	1.2437549	0.498082602	0.000255102	0.004373663	0.426565012	0.001651
11	708.1311242	8.941276802	4.820912921	0.226374843	0.060180811	17.79143834	12.58785335	0.600929	4.469111843	0.6006152	0.241359363	7.97303E-05	0.001013424	0.089224425	0.000767
12	1182.149881	20.32849371	12.1486833	0.598602522	0.092971567	19.67232806	15.80071474	0.929647	3.940922895	0.3183505	0.127580315	0.000182393	0.006874268	0.075553461	0.000906
13	2774.271886	72.00647649	10.22745989	0.150853917	0.007674639	4.98658498	1.25424686	2.064636	2.181264446	2.0781428	0.83447347	7.18088E-05	0.003150673	0.040336985	0.002995
14	735.7074558	6.149475783	1.749002677	0.06014557	0.004714645	22.31424119	1.999646984	6.232009	8.391316586	0.3664636	0.146590505	1.87598E-05	0.002638391	0.065833149	0.002349
15	870.2084611	0.09641052	0.029637912	0.000856298	0.00319399	6.203973211	5.893064228	0.045648	0.145880115	0.0041495	0.001696435	1.06743E-05	0.00115437	0.00252979	5.97E-05
16	297.5646767	4.981746672	1.443727469	0.05426993	0.008490416	5.682897528	2.299366044	2.022846	2.318563457	3.9919691	1.600770008	9.3013E-05	0.013395539	0.052433575	0.001228
17	1568.044092	12.97358466	6.679034366	0.333396801	0.046897147	57.36158628	17.2036695	2.285452	6.953966985	0.5901371	0.26329413	0.000107958	0.004003882	0.030769416	0.002799

In this way, were able to bridge our two modules. We were able to find out the nutritional score for each value, and the results are as follows:

Now we can finally bridge the two models! We passed the nutritional values of each cluster to the ingredients to nutri-score model and got the predicted nutri-scores for each cluster. The predicted nutri-scores were surprisingly good! Most of the clusters got a nutri-score that seemed to fit the main ingredients of the cluster. Here are the Nutri-Scores predicted for the clusters:

```
['e' 'd' 'c' 'd' 'e' 'c' 'b' 'c' 'b' 'b' 'd' 'c' 'd' 'e' 'a' 'b' 'c' 'd']
```

The 'a' score corresponds to the cluster number 14, which contains mainly beans.

We have 'b' scores for clusters 6, 8, 9 and 15 which contain mainly: berries (6), chicken (8), water (9) and **alcohol** (15).

Clusters with 'c' scores are 2, 5, 11, 16 which contain mainly: eggs (2), cocktails (5), milk (11) and **tomato based dishes** (16).

For 'd' scores we have clusters 1, 3, 10, 13 and 17 which contain mainly: **almonds** (1), **water & nuts** (3), cheese (10), oil (13) and baked goods (17).

Finally for 'e' scores we have clusters 0, 4 and 12 which contain mainly: butter (0), baked products and sweets (4) and other desserts (12).

We are very satisfied with these results! First, they show that we can successfully bridge the two models. Second, the predicted nutri-scores seem to make sense given the main ingredients of each cluster. This shows that our approach of mapping ingredients to nutritional values and then predicting nutri-scores is valid. Indeed, apart from some outliers (the ones in bold), the predicted nutri-scores seem to align well with our expectations based on the main ingredients of each cluster.

Figure 13: Final Nutri-Score predictions for the ingredient clusters

12. Brief description of who did what

Arne:

- "Ingredients_Data.ipnyb" script for meals dataset preprocessing
- Assisting Ali with clustering, CNN and bridging the datasets
- Creation of documentation structure in GitHub
- Team management
- Report: Introduction, Related work, Dataset Description and Analysis (Dataset descriptions and preprocessing of meals dataset)

Ali:

- "img_kmeans_CNN.ipynb" for clustering meals and the CNN for images.
- "macromapping.ipynb" for bridging the two modules.
- Assisting Arne with Preprocessing
- Report: K-Means and CNN parts of the Experimental setup, Results, and Discussion sections.

Hannah:

- Preprocessing of Open Food Facts dataset (manual removal of empty and redundant features)
- Visualisation work using graphs during preprocessing and data exploration
- Initial Keras MLP model implementation
- Implementation and finalisation of Scikit MLP model, including hyperparameter tuning and K-fold cross-validation
- Visualising results from MLP model using graphs
- Fleshing out and completing the Experimental Setup section of the report
- Nutri-Score + Real-World Implications subheadings of the Discussion section
- Conclusion section

Karol:

- "Data_Preprocessing.ipnyb" script for Open Food Facts dataset
- "Random_Forrest.ipnyb" script for Open Food Facts dataset
- Preprocessing steps of the Open Food Facts dataset in the Dataset description and analysis of the report
- Nutri-score prediction section of the report
- Team management and task delegation

Wajid:

- Tried different preprocessing approaches locally and found manual manipulation of the dataset was the best approach which Hanah did for removing empty and redundant data
- Documentation creation and maintenance across multiple weeks
- Team collaboration and coordination
- Single-layer Perceptron baseline model implementation for Nutri-Score classification
- Model validation using train-test split with stratified sampling and StandardScaler normalization
- Performance evaluation with classification reports, confusion matrices, and per-class metrics analysis
- Feature importance analysis identifying key nutritional predictors through weight magnitudes
- Visualization of results using confusion matrix, bar charts, etc
- Resolved conflicts and Merged image_models and ai_food_images branches into main branch.
- Report: Proofread entire report to check for mistakes and ensure each section matched actual work